



Sistemas Distribuídos

Professor: Rodrigo da Rosa Righi

Contato: rrrighi@unisinus.br

Agenda

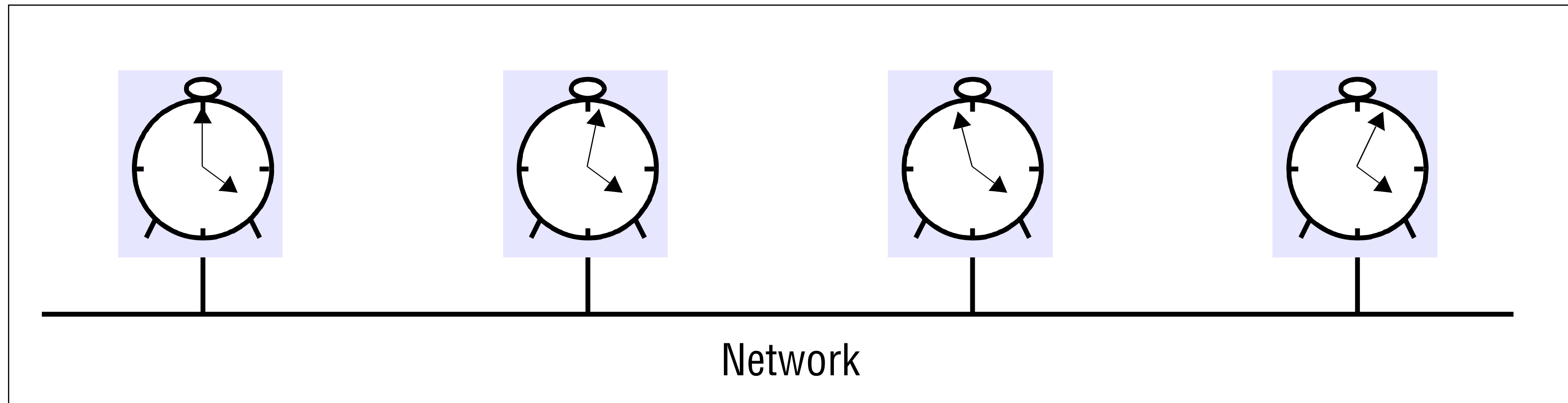
- * Sincronização de Relógios
 - * Algoritmo de Cristian
 - * Algoritmo de Berkeley
- * Relógios Lógicos
 - * Timestamps de Lamport
 - * Multicast Totalmente Ordenado
 - * Vetor de Timestamps
- * Estado Global

Sincronização de Relógios

- ❖ Em um **ambiente centralizado**, tempo **não é ambíguo**. Quando um processo necessita saber o tempo, ele faz uma chamada de sistema e o Kernel devolve o resultado.

Sincronização de Relógios

❖ Num **sistema distribuído**, alcançar um acordo quanto ao tempo **não é uma tarefa trivial**



Sincronização de Relógios

Exemplo
tradicional

Funcionamento do sistema make.

O comando make examina os tempos de modificação de todos os arquivos fontes e os objetos.

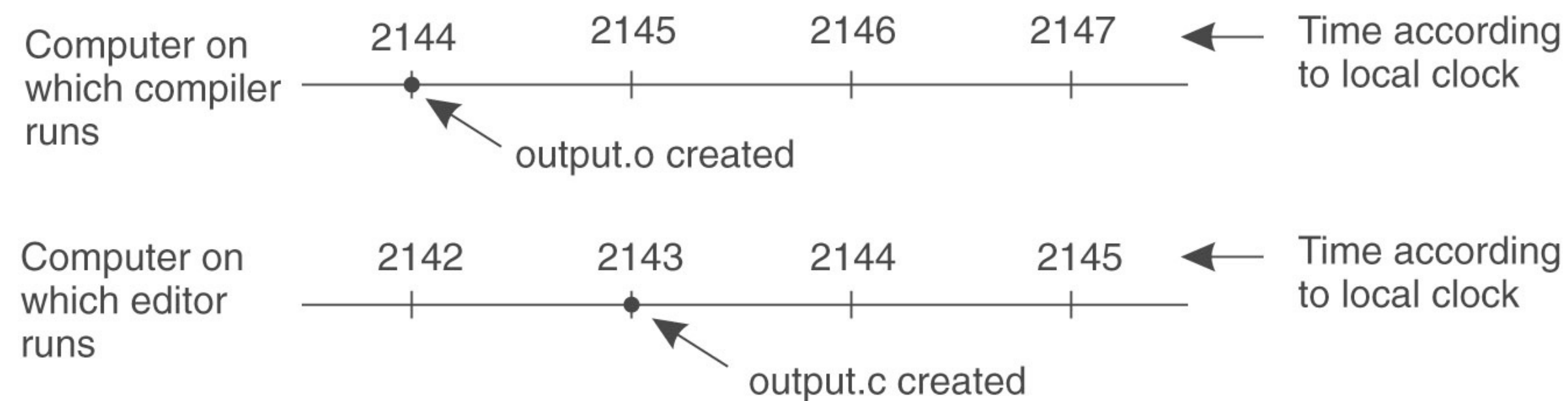
Se um arquivo fonte tem tempo 2151 e o seu correspondente objeto tem tempo 2150, make sabe que o arquivo fonte foi modificado e o objeto necessita ser recompilado.

Sincronização de Relógios

❖ Imagine um sistema distribuído onde **não há um acordo quanto ao tempo**

Suponha um objeto com tempo 2144 e um arquivo fonte modificado posteriormente mas seu tempo é marcado como 2143 por causa que a máquina que o escreveu está com o clock atrasado.

O make pode resultar num crash e o programador terá grandes problemas para resolver.



Sincronização de Relógios

❖ Questões quanto ao tempo do computador

O tempo de um computador é precisamente fornecido por cristais de Quartz. Na medida que múltiplos CPUs foram introduzidos, cada qual com seu próprio clock, a situação de acordo também tornou-se um desafio.

É impossível garantir que cristais em diferentes computadores executem exatamente na mesma frequência.

Sincronização de Relógios

Questões quanto ao tempo do computador

Na prática, quando um sistema possui n computadores, todos os n cristais rodam com taxas ligeiramente diferentes, causando gradualmente uma perda de sincronização e retornando diferentes valores quando lidos.

Esta diferença é chamada de clock Skew.

Em sistemas real-time, o clock atual é de suma importância. Por questões de eficiência e redundância, múltiplos relógios fixos são geralmente considerados e acarretam em 2 problemas:

Como sincronizá-los com o clock do mundo real

Como sincronizar um com o outro

Sincronização de Relógios

Como manter o tempo sincroni- zado

Atualmente, utiliza-se como valor de tempo oficial o UTC - Coordinated Universal Time.

É baseado no tempo atômico e pode incluir muitas vezes um leap-second para manter-se sincronizado com o tempo observado na astronomia. Sinais UTC são sincronizados e repassados em mensagens broadcast regularmente de estações de rádio em terra e satélites, cobrindo grande parte do globo.

Por exemplo, nos EUA, estações de rádio fazem broadcast de sinais de tempo em várias frequências de ondas curtas.

Receptores estão disponíveis comercialmente. Comparado com o tempo UTC perfeito, sinais recebidos de estações em terra possuem uma precisão na ordem de 0.1 a 10 milisegundos. Sinais recebidos de um GPS possuem uma precisão de 1 microsegundo. Computadores com receptores sincronizam seus clocks baseados nesses sinais. Computadores podem atuar como clientes do serviço NTP (Network Time Protocol).

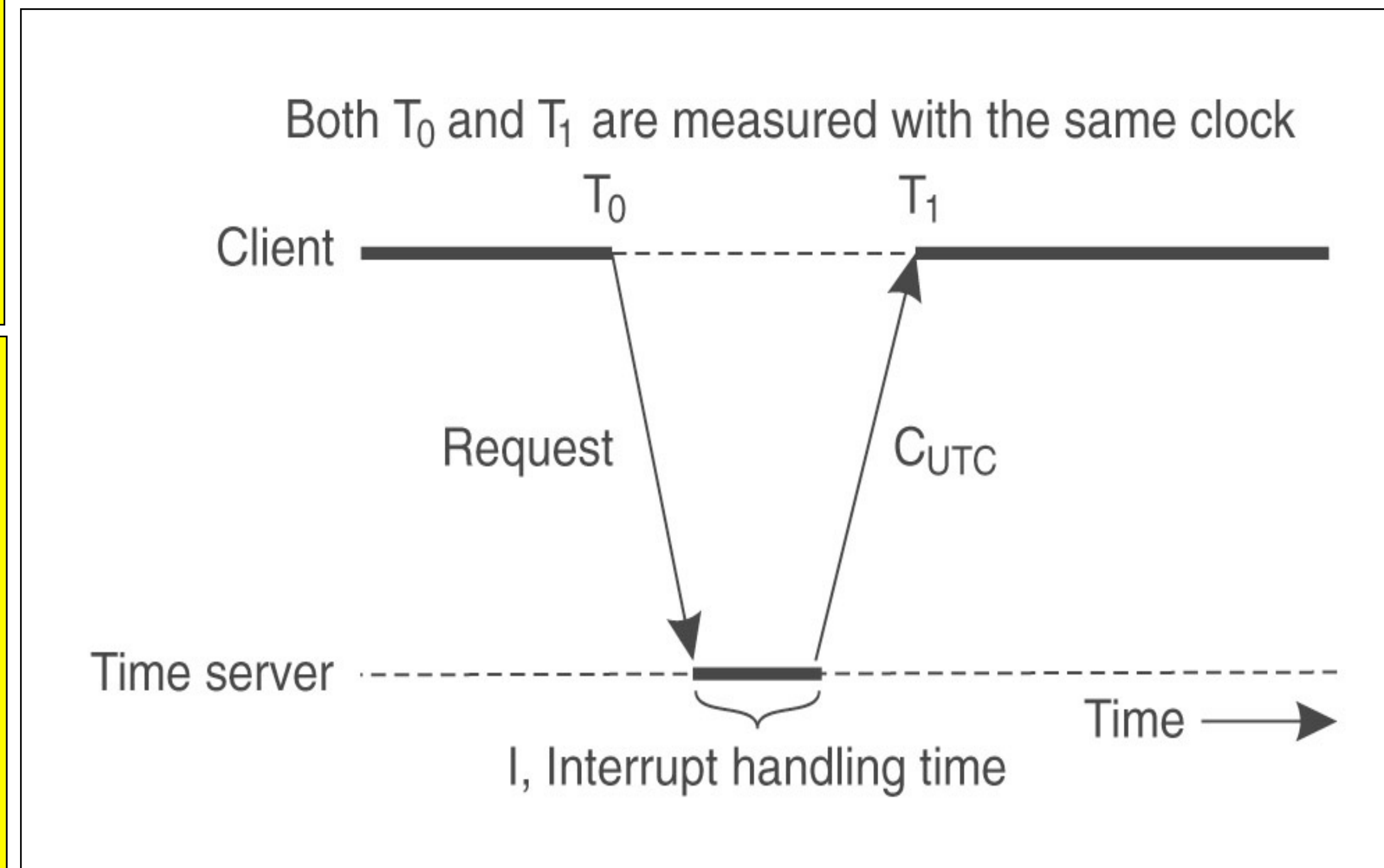
Sincronização de Relógios



Algoritmo de Cristian

Algoritmo viável para uma máquina que tenha um servidor de tempo e o objetivo é ter todas as outras máquinas sincronizadas com ela.

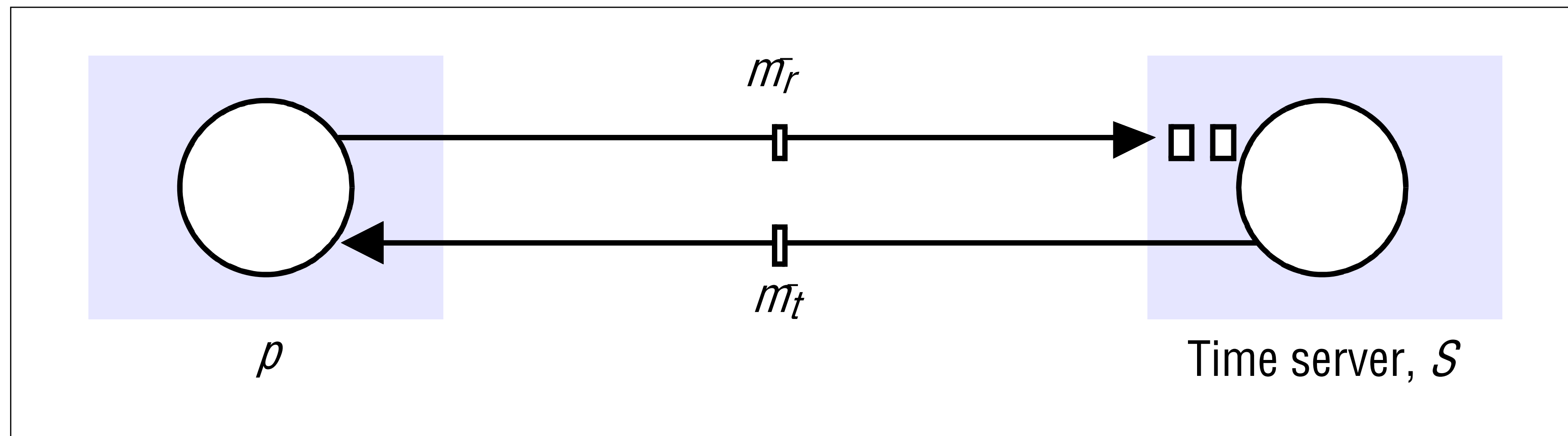
Periodicamente, cada máquina envia uma mensagem para o servidor de tempo perguntando pelo tempo corrente. Esse último responde o mais rápido possível com uma mensagem com Cutc (clock segundo Universal Coordinated Time).



Sincronização de Relógios



Algoritmo de Cristian



Arquitetura Cliente Servidor

Sincronização de Relógios



Algoritmo de Cristian

Quando uma máquina captura a resposta, ela simplesmente estabelece seu clock com Cutc. Entretanto, esse algoritmo possui 2 problemas:

O tempo nunca estará acertado com relação ao do servidor. Uma possibilidade é acrescentar alguns milissegundos ao tempo. A idéia é empírica e consiste em aumentar ou diminuir um certo tempo em milissegundos ao tempo do servidor.

O tempo de resposta pode variar de acordo com o congestionamento da rede. A idéia do algoritmo é então medir o atraso da rede. O transmissor deve medir o tempo entre enviar uma requisição ao servidor de tempo e a chegada da resposta. Assim captura os tempos t_0 e t_1 e calcula o seu tempo:

$$\text{Tempo} = \text{tempo do servidor} + (t_1 - t_0)/2$$

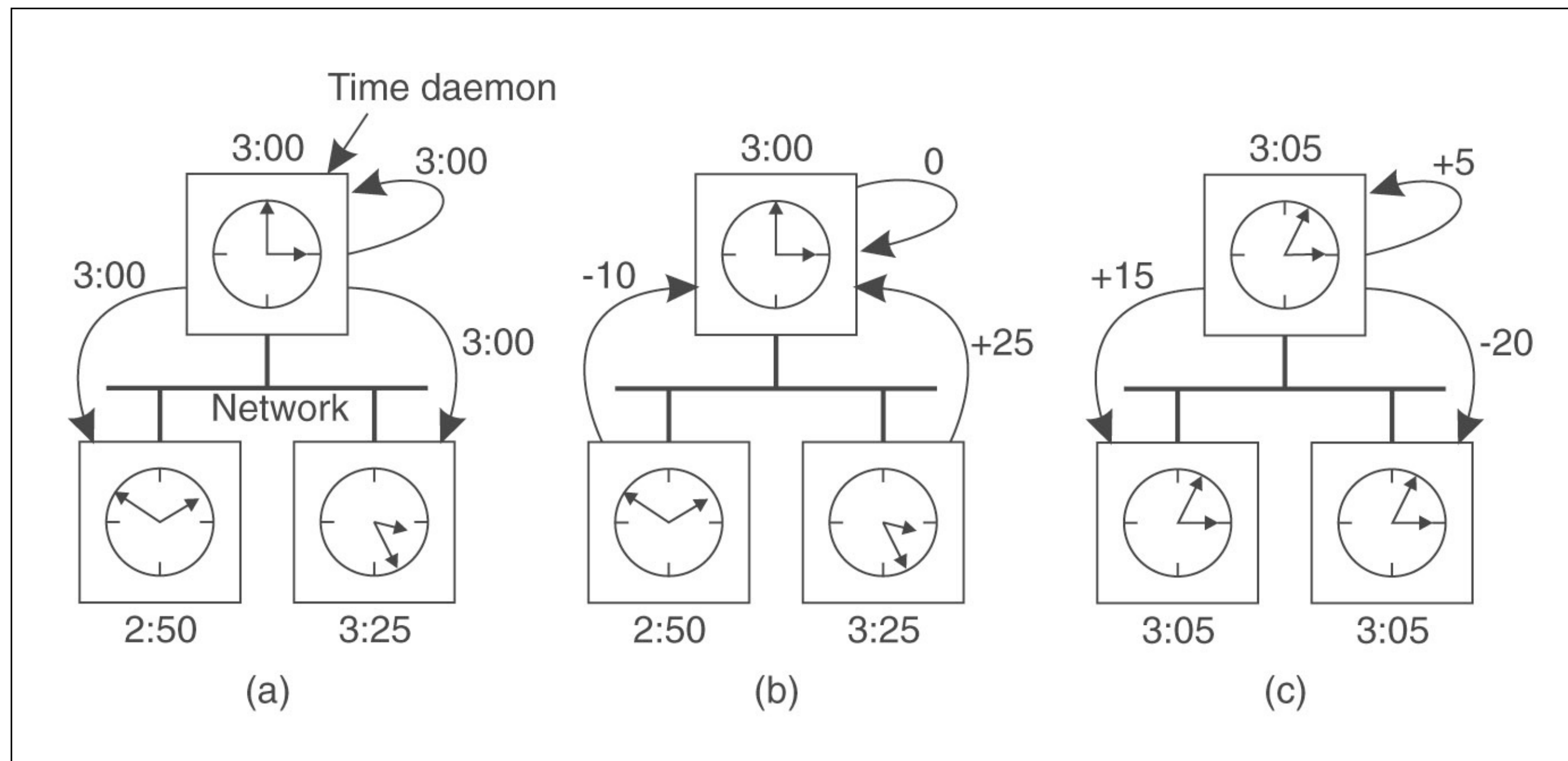
Para melhorar as medidas, Cristian sugere a tomada de vários tempos de rede e a realização de médias. Mesmo assim, mensagens muito rápidas ou muito lentas podem interferir negativamente na média.

Sincronização de Relógios

Algoritmo de Berkeley

No algoritmo de Cristian, o servidor é passivo. Demais máquinas periodicamente o questionam sobre o tempo.

Em sistemas Berkeley UNIX, a abordagem oposta é tomada. O servidor de tempo é ativo, verificando cada máquina de tempos em tempos perguntando a elas qual o tempo que elas possuem. Baseado nas respostas, ele computa uma média de tempo e diz para todas as máquinas para avançarem seus tempos para um novo tempo ou para retrocederem uma determinada quantia.



(a) Servidor de tempo pergunta para todas as máquinas sobre seus valores de clock; (b) as máquinas respondem; (c) o servidor envia a cada uma um ajuste de clock

Sincronização de Relógios



Algoritmo de Berkeley

O servidor de tempo deve ser setado manualmente pelo operador periodicamente

Ideia geral

Servidor pergunta o tempo, máquinas respondem, servidor faz uma média e envia o novo tempo para as máquinas

Desvantagem de ambos algoritmos de Cristian e de Berkeley

Ambos são altamente centralizados

Sincronização de Relógios



NTP - Network Time Protocol

Um dos algoritmos mais empregados na Internet é o NTP - Network Time Protocol. Ele é conhecido por uma precisão em redes WAN em um intervalo de 1 a 50 milisegundos.

NTP - Fornece um serviço que habilita clientes na Internet a serem sincronizados precisamente com o UTC. NTP também possui um módulo de segurança para proteção contra interferência acidental ou maliciosa. Clientes se ressincronizam frequentemente para corrigir distorções em seus clocks.

Existe uma árvore de servidores NTP, onde mais perto do topo da árvore estão os servidores mais confiáveis.

Relógios Lógicos

Para uma classe de algoritmos, o que importa é a **consistência interna do clocks**, e não o fato que eles estejam perto do tempo real. Para esse tipo de algoritmo são usados **clocks lógicos**.

Lamport
(1978) mostrou
que a
sincronização
de relógios é
possível

Dois processos podem não concordar a respeito de que tempo é num determinado instante, mas eles devem **concordar quanto a ordem dos eventos ocorridos até então**.

Por exemplo: no **make**, o que conta é se o arquivo fonte é mais velho ou mais novo que o arquivo objeto, e não o tempo absoluto de criação deles.

Relógios Lógicos



Timestamps de Lamport

Lamport definiu a relação **aconteceu-antes**. A expressão $a \rightarrow b$ é lida como “ a aconteceu antes que b ” e significa que os processos concordam que o evento a ocorre, e então depois, o evento b acontece. Isso leva a duas situações importantes:

Se a e b são eventos de um mesmo processo, e a ocorre antes que b , então $a \rightarrow b$ é verdadeiro

Se a é um evento de uma mensagem enviada por um processo, e b uma mensagem recebida por outro, então $a \rightarrow b$ é também verdadeiro.

Uma mensagem não pode ser recebida antes que enviada, ou ainda, no mesmo tempo que foi enviada pois leva um tempo finito para chegar.

Relógios Lógicos



Timestamps de Lamport

Aconteceu-antes é relação **transitiva**, de maneira que $a \rightarrow b$ e $b \rightarrow c$, então $a \rightarrow c$.

Se dois eventos, x e y , acontecem em diferentes processos que não trocam mensagens, então $x \rightarrow y$ **não é verdadeiro**, nem $y \rightarrow x$. Tais eventos são ditos concorrentes, uma vez que nada pode ser dito quando os eventos acontecem.

Devemos medir o tempo para todo evento. Por exemplo, para o evento a , o valor $C(a)$ implica no tempo que todos os processos concordam. Se $a \rightarrow b$, então $C(a) < C(b)$.

Relógios Lógicos



Timestamps de Lamport

A solução de Lamport segue diretamente a relação **aconteceu-antes**. Portanto, **cada mensagem carrega o tempo de transmissão de acordo com o clock do transmissor**. Quando a mensagem chega e o clock do receptor mostra um valor menor que aquele da mensagem enviada, o receptor aumenta seu clock de maneira que seja maior que o do transmissor.



A cada dois eventos, o clock deve aumentar pelo menos uma vez. Se um processo envia ou recebe duas mensagens sucessivamente, ele deve avançar o seu clock de pelo menos 1 a cada evento.

Relógios Lógicos



Timestamps de Lamport

Método para setar o tempo dos eventos

Se a acontece antes que b
em um mesmo processo,
 $C(a) < C(b)$

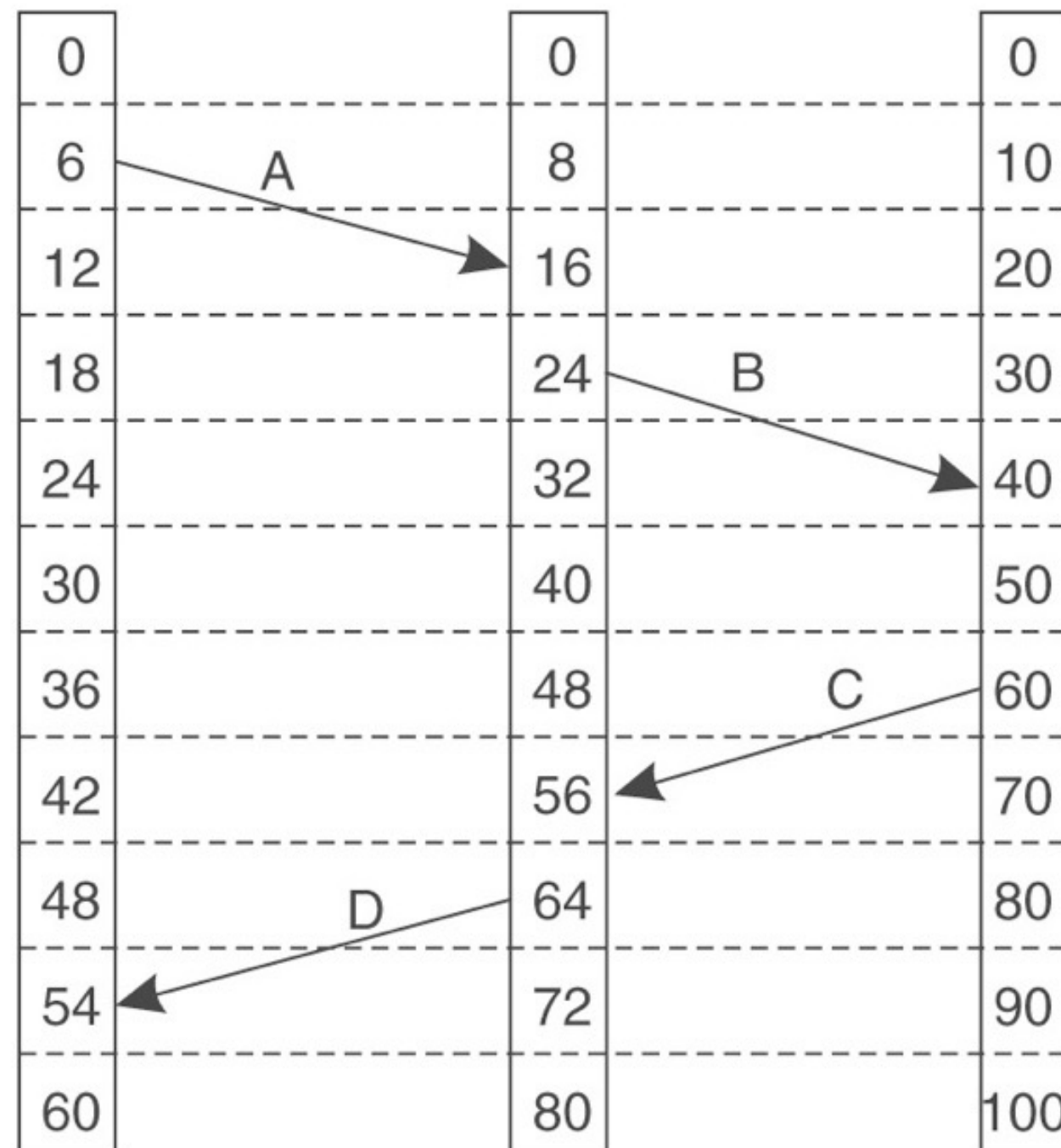
Se a e b representam o envio e a
recepção de uma mensagem,
respectivamente, então $C(a) < C(b)$

Para todos eventos distintos
 a e b , então $C(a) \neq C(b)$

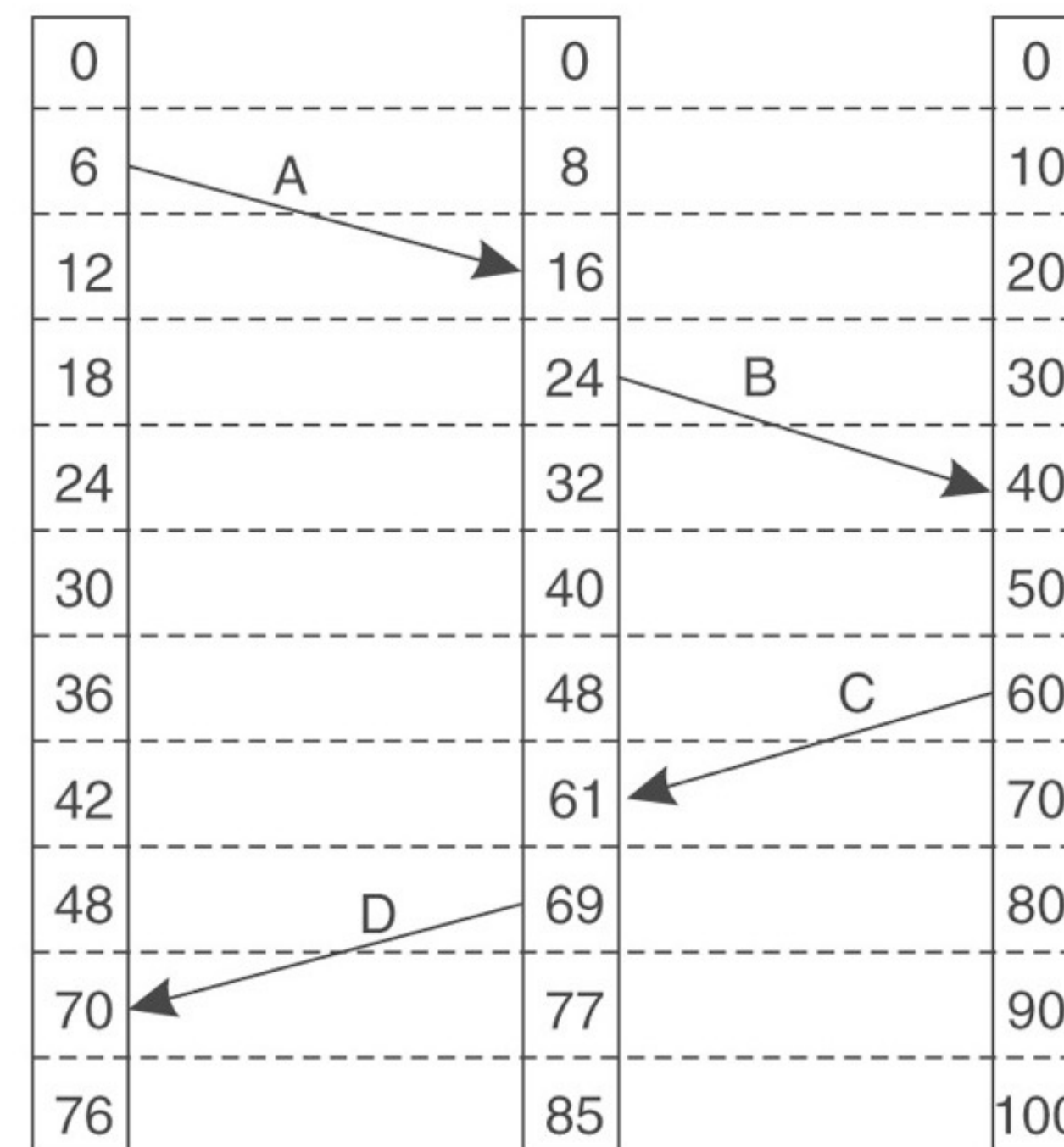
Relógios Lógicos



Timestamps de Lamport



(a)



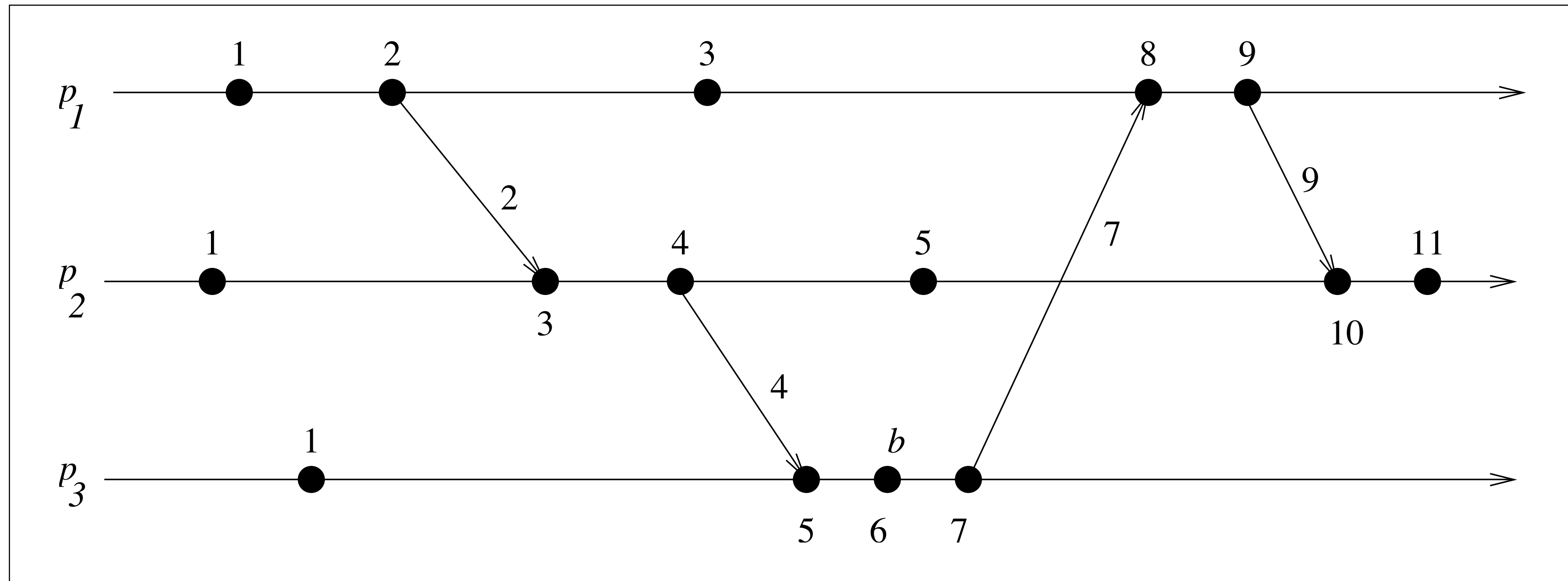
(b)

(a) Três processos, cada qual com seu próprio clock. Os clocks executam em taxas de avanço diferentes; (b) Correção dos clocks usando o algoritmo de Lamport

Relógios Lógicos



Timestamps de Lamport



Relógios Lógicos

Vetor de Timestamps

Timestamps de Lamport lidam com uma situação onde todos os eventos no sistema distribuído são totalmente ordenados com a propriedade de que se a aconteceu antes que b , então a também será posicionada na ordem antes que b , ou seja, $C(a) < C(b)$.

Entretanto, nada pode ser dito entre a relação entre dois eventos a e b simplesmente comparando seus valores de tempo $C(a)$ e $C(b)$. Em outras palavras, se $C(a) < C(b)$, não necessariamente implica que a aconteceu antes que b ($a \rightarrow b$).

Timestamps de Lamport não capturam causalidade. Existem eventos que devem ocorrer antes que outros (Exemplo: para o carro dar partida, primeiramente é necessário colocar a chave na ignição). Consequentemente, existem situações onde a relação causal deve ser mantida dentro de um grupo de processos.

Relógios Lógicos

Vetor de Timestamps

Causalidade por ser capturada usando vetor de timestamps. Um vetor de timestamps $VT(a)$ mapeado para um evento a tem a propriedade que se $VT(a) < VT(b)$ para algum evento b , então o evento a é conhecido como precedência causal para o evento b . Vetor de timestamps são construídos tomando que cada processo P_i mantenha um vetor V_i com as seguintes propriedades:

$V_i[i]$ é o número de eventos que ocorreram em P_i

Se $V_i[j] = k$, então P_i conhece k eventos ocorridos em P_j



Relógios Lógicos

Vetor de Timestamps

A primeira propriedade é mantida incrementando $V_i[i]$ a cada ocorrência de um novo evento que acontece no processo P_i . A segunda propriedade é mantida retornando vetores com mensagens que são enviadas.

Quando P_i envia mensagem m , ele envia junto o seu vetor corrente com um timestamp.



Multicast totalmente ordenado

O algoritmo de **multicast** totalmente ordenado também faz uso dos relógios de Lamport e da ordenação total. Ele se faz necessário quando precisamos garantir a mesma sequência de eventos em todos os processos do sistema, como por exemplo, em uma atualização de **banco de dados** replicado em diversas máquinas.

O algoritmo funciona da seguinte forma:

1. Um processo cria uma mensagem contendo sua marca temporal atual e então faz multicast para todos os outros processos do grupo.
2. Os contadores são atualizados no envio e recebimento das mensagens.
3. Quando uma mensagem é recebida, o processo a coloca em sua fila local de acordo com sua marca temporal e aguarda pela confirmação das mensagens enviadas.

Uma mensagem então só é de fato entregue à aplicação quando está na cabeça da fila e todas confirmações daquela mensagem foram recebidas. Ou seja, o processo garante que a marca temporal da mensagem entregue é menor que os contadores de outros processos. Além disso, desempates de marcas temporais de mensagens são feitos através de identificadores arbitrários dos processos, permitindo a ordenação total dos eventos.

Relógios Lógicos

Vetor de Timestamps

Assim, o receptor é informado sobre o número de eventos que ocorrerem em P_i . Mais importante, é que o receptor percebe **quantos eventos em outros processos aconteceram antes que P_i enviasse a mensagem m .**

Em outras palavras, timestamp vt da mensagem m informa o receptor sobre **quantos eventos em outros processos precedem m** , e assim m depende casualmente deles.

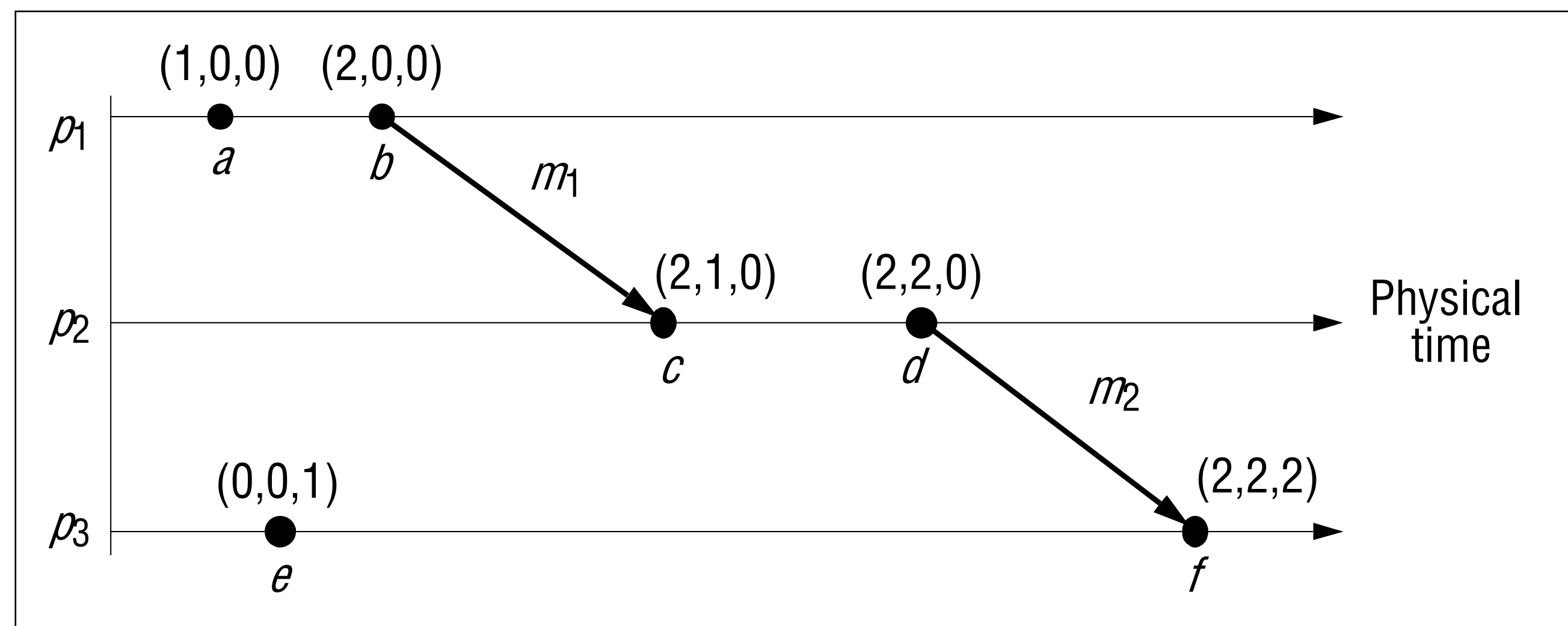


Relógios Lógicos

Vetor de Timestamps

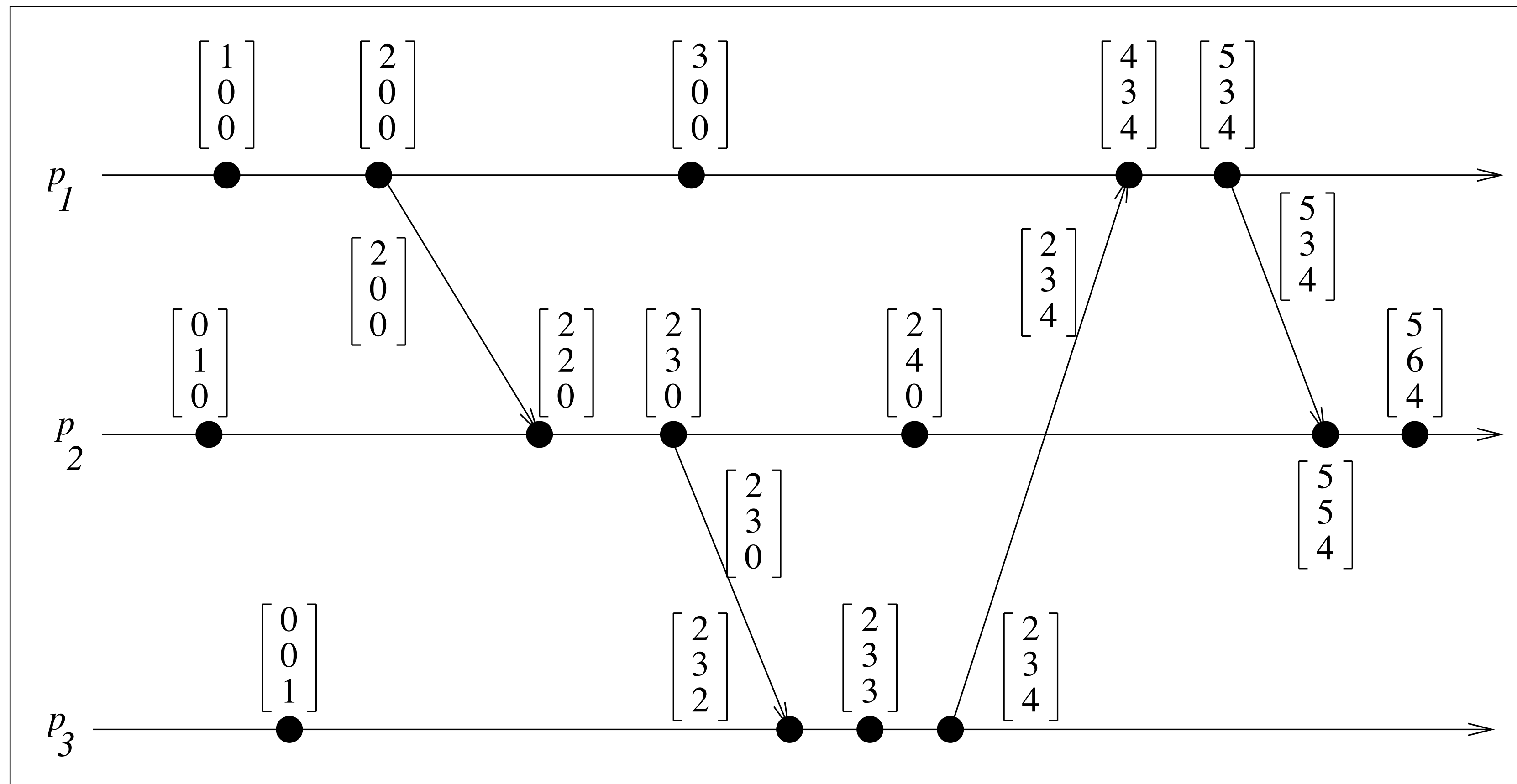
Desvantagem do uso de vetor de Timestamps se comparados com os timestamps de Lamport

Devemos transmitir o vetor em cada mensagem e o tamanho dele é proporcional ao número de estações



Relógios Lógicos

Vetor de Timestamps



Estado Global

❖ Em muitas situações, é pertinente conhecer o estado global de um sistema distribuído

Ele consiste do estado local de cada um dos processos, juntamente com as mensagens que estão em trânsito, ou seja, que foram enviadas mas não foram ainda recebidas

O estado local de um processo depende da finalidade do sistema distribuído. No caso de um sistema de banco de dados, o estado local pode consistir somente daqueles registros que formam parte de um banco de dados e exclui registros temporários usados para computações.

O estado global pode ser útil para, por exemplo, quando é conhecido que computações locais pararam e que não existem mensagens em trânsito, o sistema entrou em uma condição na qual nenhum progresso pode ser feito. Nesse caso, pode-se concluir que o sistema entrou em um **deadlock** ou que **a computação distribuída terminou corretamente**.

Estado Global

Uma maneira de capturar o estado global de um sistema distribuído é através de um snapshot distribuído

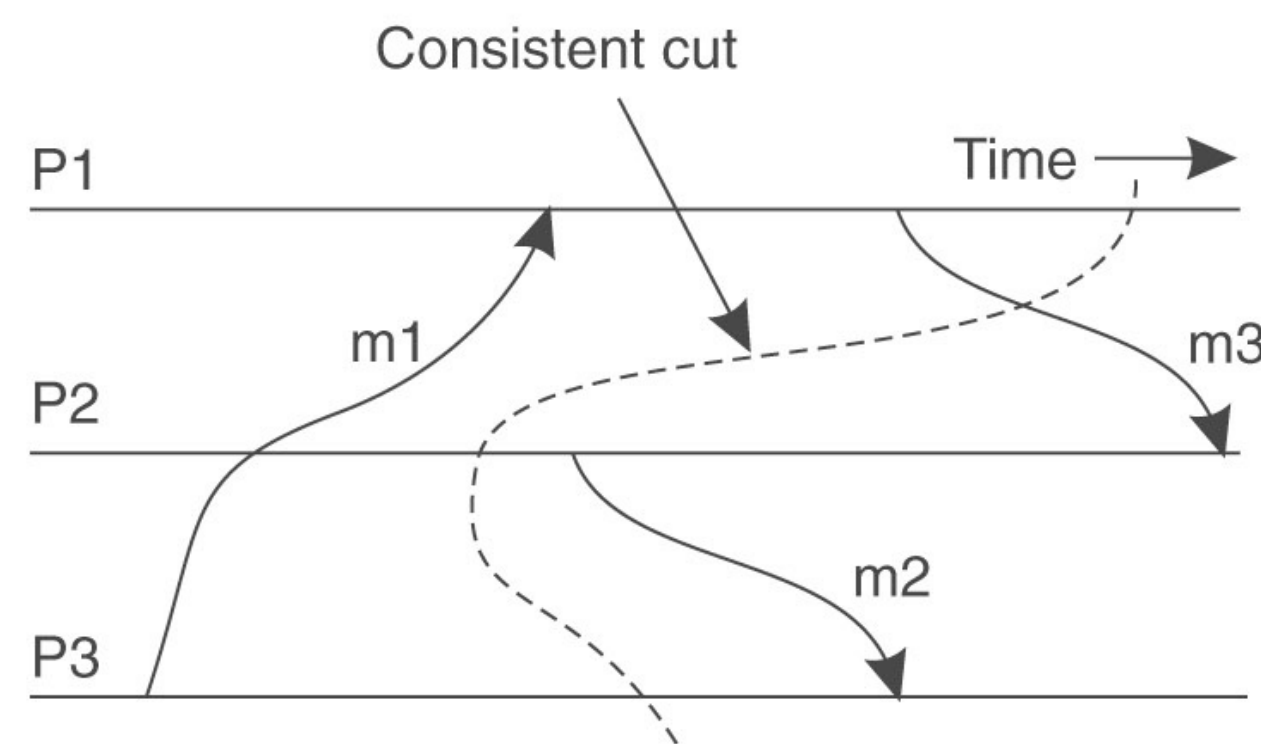
Um snapshot reflete o estado no qual o sistema distribuído pode estar.

Em particular, isso significa se gravamos que um processo P recebeu uma mensagem de outro Q , então devemos também gravar que o processo Q enviou a mensagem. Não podemos ter a recepção de uma mensagem sem a sua devida transmissão. Entretanto, uma sentença muito importante é que: podemos ter a condição de uma mensagem enviada por Q mas ainda não recebida por P .

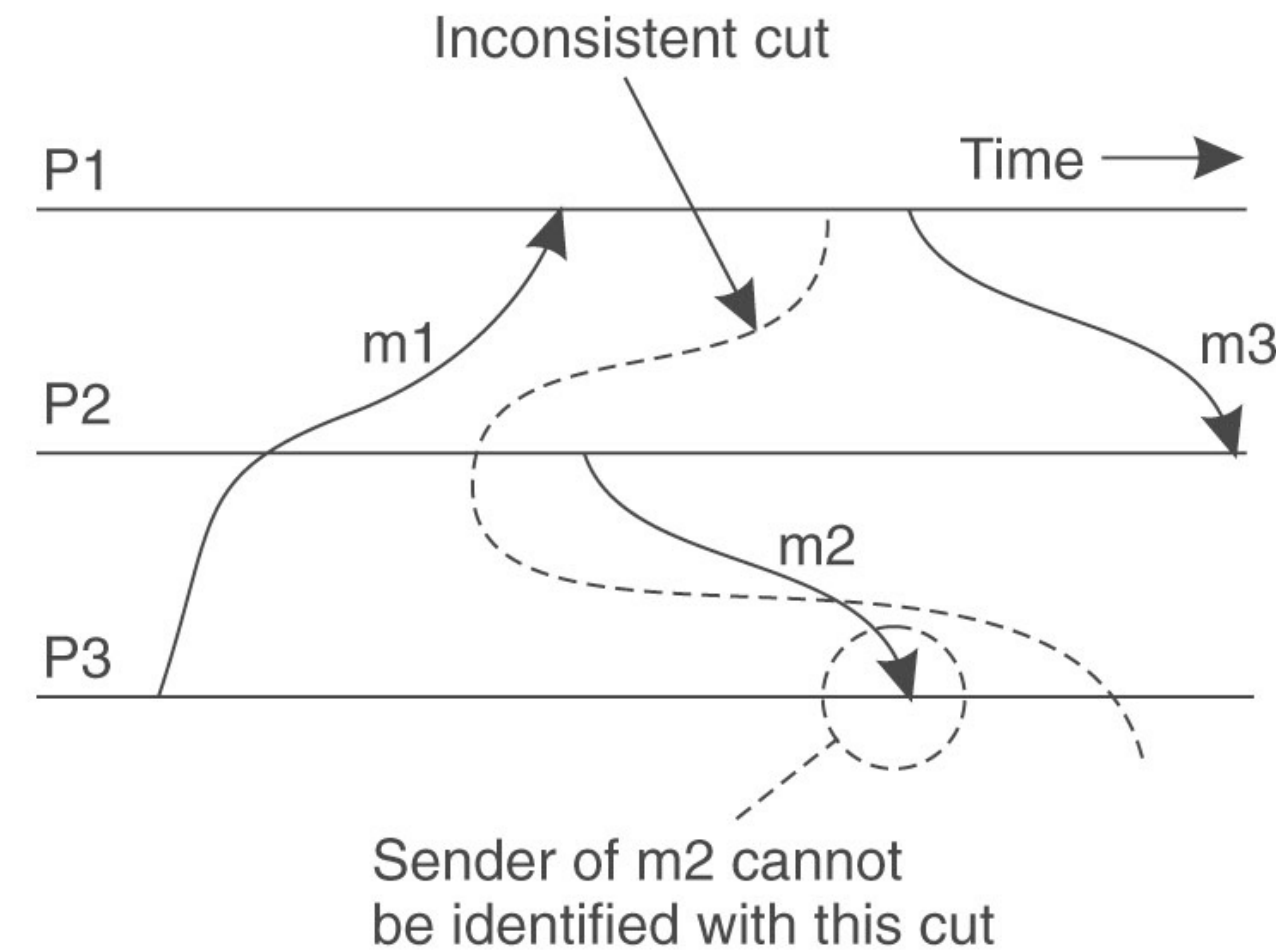
Estado Global

- ❖ A noção de estado global pode ser representada graficamente por um corte. Um **corte consistente** é mostrado em linhas pontilhadas. O corte representa o último evento que foi gravado em cada processo.

Na figura (a) pode-se verificar que todas as mensagens no receptor tiveram um evento de envio gravado no transmissor. Já na figura (b) a recepção da mensagem *m2* pelo processo *P3* foi gravada, mas o snapshot não contém o evento correspondente de envio.



(a)



(b)

(a) Corte consistente; (b) Corte inconsistente